

CHURCH-TURING THESIS

The power of Turing Machine was described by the American logician ^{Church} λ in the year 1936 in the American Mathematics Journal. The statement is known as Church's Thesis or Church Turing Thesis, stated as under;

"Any algorithmic procedure that can be carried out by a human, a team of humans, or a computer, can be carried out by Turing Machine".

Though this is not a mathematically precise statement as "Algorithmic Procedure" is not precisely defined, since the invention, enough evidence is found for its correctness and thus generally accepted (Mathematically, it can't be proved because of above constraint)

Accepting above statement, we can state that

"An algorithm is a procedure that can be executed on a TM".

The advantage of the definition is that it can be discussed about the ~~algorithmic~~ problems that can be solved algorithmically and problems (if any) that can't be solved

Recursively Enumerable Languages

Accepting a language and Recognizing a language

• Let $L \subseteq \Sigma^*$ be a language.

A Turing machine T with input alphabet Σ is said to accept L if $L(T) = L$.

T recognizes or decides L if T computes the characteristic function $\chi_L : \Sigma^* \rightarrow \{0, 1\}$.

In other words, T recognizes L if T halts in state h_a for every string x in Σ^* , producing output 1 if $x \in L$ and output 0 otherwise.

• A language L is recursively enumerable if there is a TM that accepts L , and recursive if there is a TM that accepts $\bar{L}$, and recursive if there is a TM that recognizes L . (Languages are called Turing-acceptable and Turing-decidable.)

* Every recursive language is
recursively enumerable

Proof

If a Turing Machine recognizes L , then

we can get a TM accepting L by

modifying T so that when it leaves

output 0, it enters the reject state.

• Th: If L is accepted by a non-deterministic

TM T , and every possible sequence

of moves of T causes it to halt,

then L is recursive

• Th: If L_1 and L_2 are recursively enumerable

languages over Σ , then $L_1 \cup L_2$ and

$L_1 \cap L_2$ are also recursively enumerable.

Theorem

* Th: If L is recursive, so is L' .

Proof If T is a Turing Machine recognizing L , we can make it recognize L' by:

"Interchanging the two outputs."

Unrestricted Grammar

An unrestricted or phrase-structure grammar is a 4-tuple $G = (V, \Sigma, S, P)$ where V and Σ are disjoint sets of variables and terminals respectively; S is an element of V called the start symbol; P is a set of productions of the form

$$\alpha \rightarrow \beta$$

where, $\alpha \in \beta \in (V \cup \Sigma)^*$ and α contains at least one variable.

An unrestricted Grammar Generating

$$L = \{ a^i b^i c^i \mid i \geq 1 \}$$

$$S \rightarrow F S_1$$

$$S_1 \rightarrow ABC S_1 \mid ABC$$

$$BA \rightarrow AB$$

$$CA \rightarrow AC$$

$$CB \rightarrow BC$$

$$FA \rightarrow a$$

$$aA \rightarrow aa$$

$$aB \rightarrow ab$$

$$bB \rightarrow bb$$

$$bC \rightarrow bc$$

$$cC \rightarrow cc$$

(Here, variable F is used for the left end of the string. Here, we say that A can be replaced by 'a' only when it is preceded by 'a' or F . This will avoid strings like "abcabc", if we say $A \rightarrow a$)

A typical derivation for "aa bb cc"

$$S \Rightarrow F S_1 \Rightarrow F A B C S_1 \Rightarrow F A B C \underline{A} B C$$

$$\Rightarrow F A B \underline{A} C B C \Rightarrow F A A B \underline{C} B C \Rightarrow \underline{F} A A B B C C$$

$$\Rightarrow a A B B C C \Rightarrow a a \underline{B} B C C \Rightarrow a a b \underline{B} C C$$

$$\Rightarrow a a b b \underline{C} C \Rightarrow a a b b c C \Rightarrow a a b b c c$$

Theorems

- For any ~~un~~-restricted grammar $G = (V, \Sigma, S, P)$, there is a Turing machine $T = (Q, \Sigma, \Gamma, q_0, \delta)$ with $L(T) = L(G)$
- For any recursively enumerable language $L \subseteq \Sigma^*$, there is an unrestricted grammar G generating L .

Definition — Context Sensitive Grammar

A context-sensitive grammar (CSG) is an unrestricted grammar in which every production has the form

$$\alpha \rightarrow \beta \quad \text{with} \quad |\beta| \geq |\alpha|$$

A context-sensitive language (CSL) is a language that can be generated by such a grammar.

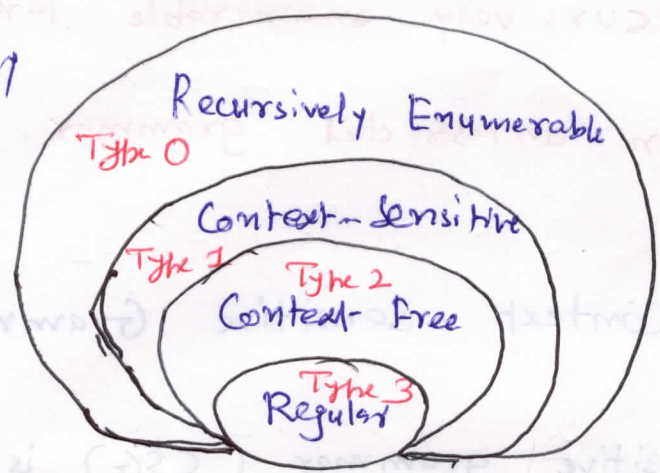
eg. C language

Theorem: Every Context Sensitive Language is Recursive.

- Not all languages are Recursively Enumerable

If L_1 and L_2 are recursive languages,
then $L_1 \cup L_2$ and $L_1 \cap L_2$ are also
recursive.

CHOMSKY HIERARCHY



Grammar	Languages (grammars)	Automaton	Production rules	Example
Type-0	Recursively Enumerable	Turing Machine	$\alpha \rightarrow \beta$ $\alpha, \beta \in (V \cup \Sigma)^*$ α , contains a variable	
Type-1	Context-Sensitive	Linear-bounded Non-Deterministic Turing Machine	$\alpha \rightarrow \beta$ $(\alpha, \beta \in (V \cup \Sigma)^*,$ $ \beta \geq \alpha , \alpha$ contains a variable	
Type-2	Context-Free	Pushdown Automaton		
Type-3	Regular	Finite State Automaton		

Production Rules for Type 2

$$A \rightarrow \alpha$$
$$A \in V, \alpha \in (V \cup \Sigma)^*$$

Production Rules for Type 3

$$A \rightarrow aB, A \rightarrow a$$
$$A, B \in V, a \in \Sigma$$

(7)
Example for Type ~~0/1~~ (PSG) Languages
(Zero)

$$L = \{a^n b^n c^n \mid n > 0\} \leftarrow \begin{array}{l} \text{Recursively} \\ \text{Enumerable} \end{array}$$

It's also a CSG (CSL) — Type ~~1~~.

Example of Type 2 Language (CFL)

$$L = \{a^n b^n \mid n > 0\}$$

Example of Type 3 Language (Regular)

$$L = \{a^n \mid n \geq 0\}$$